Project 6 Work Report
DeepInverse Wrapper for SIRF/CIL PET Operators

Workspace: `/home/fotis/hackathon`

March 11, 2026

1 Objective

The assigned topic was Project 6:

> DeepInverse wrapper for PET operators, wrapping e.g. parallelproj/SIRF/CIL/pytomography. The goal is to create a DeepInverse physics wrapper so that PET acquisition operators can be used natively with DeepInverse algorithms and tooling.

The target shape suggested by the organisers was a DeepInverse class similar to:

```
class deepinv.physics.EmissionTomographyWithSIRF(deepinv.physics.LinearPhysics):
...
```

The required technical elements were:

- a DeepInverse-facing wrapper for PET operators,
- an end-to-end example,
- tests covering adjointness and normalisation,
- compatibility with the broader DeepInverse ecosystem.

2 Delivered Work

The work that was completed is listed below by job rather than by clock time.

2.1 Environment and Safety Setup

- Preserved the existing known-good SIRF PET/STIR Python build and did not replace it in place.
- Set up backup and note-taking workflow in the live workspace.
- Created a dedicated CPU-only machine learning environment with `torch` and `deepinv`.
- Created a separate CIL environment with `cil`, `torch`, `deepinv`, `astra-toolbox`, and `ccpi-regulariser`.
- Built a side-by-side local SIRF Python overlay for the CIL environment so that `cil`, `sirf.STIR`, `torch`, and `deepinv` can run in one interpreter.

2.2 Core Wrapper Implementation

The main code now provides two complementary wrappers:

- `EmissionTomographyWithSIRF` in `shared/sirf_deepinv_bridge/physics.py`, a DeepInverse-facing `LinearPhysics` wrapper around a SIRF PET acquisition model.
- `SIRFLinearOperatorCIL` in `shared/sirf_deepinv_bridge/cil_operator.py`, a CIL `LinearOperator` wrapper around the same class of SIRF operator.

The wrapper implementation includes:

- conversions between SIRF containers and torch tensors,
- conversions between SIRF containers and CIL containers,
- autograd-compatible forward use,
- explicit operator norm estimation,
- explicit operator normalisation support,
- a compatibility alias `SIRFLinearPhysics` for earlier code paths.

2.3 End-to-End Artifacts

Several runnable artifacts were produced:

- `wrapper_demo.py`: basic PET/STIR wrapper validation.
- `quickcheck.py`: compact Project 6 validation entrypoint.
- `cil_data_demo.py`: CIL built-in data demo with DeepInverse denoiser wrapping.
- `cil_operator_demo.py`: direct CIL wrapper demo around SIRF.
- `phantom_compare.py`: synthetic phantom versus reconstruction benchmark with saved figures and metrics.

2.4 Tests and Checks

The following validation jobs were completed:

- PET/STIR smoke test for adjointness, norm estimation, and normalisation scaling.
- DeepInverse smoke test for the differentiable path.
- CIL-data smoke test using built-in CIL example data.
- CIL/SIRF operator smoke test using relative dot error.
- Unified-runtime quickcheck in a single `cil + sirf + deepinv` interpreter.

3 Files Produced or Updated

Path	Purpose
`shared/sirf_deepinv_bridge/physics.py`	DeepInverse-facing SIRF wrapper, including normalisation support.
`shared/sirf_deepinv_bridge/cil_linear.py`	CIL `LinearOperator` wrapper around SIRF acquisition model.
`shared/sirf_deepinv_bridge/cil_data.py`	CIL data helper and DeepInverse denoiser proximal wrapper.
`project_06_deepinverse_wrapper_sirf_cil/wrapper_demo.py`	Basic PET/STIR wrapper demo.
`project_06_deepinverse_wrapper_sirf_cil/quickcheck.py`	Main validation checkpoint for Project 6.
`project_06_deepinverse_wrapper_sirf_cil/cil_data_demo.py`	CIL data demo using ASTRA and DeepInverse.
`project_06_deepinverse_wrapper_sirf_cil/CIL_operator_demo.py`	Demo of the CIL operator and SIRF.
`project_06_deepinverse_wrapper_sirf_cil/phantom_compare.py`	Synthetic phantom comparison, error, and metrics generation.
`tests/test_pet_bridge_smoke.py`	PET smoke test for adjointness and normalisation.
`tests/test_cil_data_smoke.py`	CIL data smoke test.
`tests/test_cil_sirf_operator_smoke.py`	CIL/SIRF wrapper dot-product test.
`submission/project_06_wrapper/draft_DRAFT.md`	Draft submission text.
`submission/project_06_wrapper/README.md`	Bundle overview.

4 Validated Runtime States

4.1 SIRF and DeepInverse CPU Runtime

The PET/STIR runtime is currently usable with:

```
cd /home/fotis/hackathon/prior_10.3.26
bash /home/fotis/hackathon/live_work/use_sirf_deepinv_cpu.sh python \
    project_06_deepinverse_wrapper_sirf_cil/quickcheck.py --skip-save
```

Validated results:

- wrapper class: `EmissionTomographyWithSIRF`
- adjointness error: 7.068786×10^{-7}
- estimated operator norm: 139.329361
- normalisation error: 0.0

4.2 Unified CIL + SIRF + DeepInverse Runtime

The unified runtime is currently usable with:

```
cd /home/fotis/hackathon/prior_10.3.26
bash /home/fotis/hackathon/live_work/use_cil_sirf_deepinv_env.sh \
    project_06_deepinverse_wrapper_sirf_cil/quickcheck.py --skip-save
```

Validated results:

- wrapper class: `EmissionTomographyWithSIRF`
- adjointness error: 4.585187×10^{-7}
- estimated operator norm: 139.329361
- normalisation error: 0.0

The CIL/SIRF operator smoke test produced relative dot error 6.440101×10^{-8}.

5 Deep Learning Content

Project 6 did require DeepInverse compatibility, but not necessarily dataset training.

The implemented work demonstrates:

- compatibility with DeepInverse abstractions through `LinearPhysics`,
- use of `TVDenoiser`,
- use of a Deep Image Prior reconstruction path,
- compatibility with CIL-side operator workflows.

What has **not** been delivered in the current state:

- training of a learned model on a PET dataset,
- diffusion sampling or self-supervised training,
- CUDA-accelerated transfer and execution validation.

Therefore, the current result is best described as a DeepInverse-compatible physics wrapper and demo package, not as a learned-model training contribution.

6 Synthetic Phantom Benchmark

To obtain a true reference for visual and quantitative comparison, a synthetic benchmark was built using the bundled PET phantom:

`/home/fotis/hackathon/11.3.26/SIRF/data/examples/PET/test_image_PM_QP_6.hv`

This phantom was forward-projected through the same SIRF PET acquisition model and then reconstructed with the Project 6 PnP path. The following files were produced:

- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/phantom.png`
- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/reconstruction.p`
- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/abs_error.png`
- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/phantom_reconstr`
- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/phantom_reconstr`
- `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phantom_compare/metrics.json`

Method	**RMSE**	**Correlation**
Backprojection	0.4378408	0.7266142
DeepInverse `A_dagger` with CG (60 iterations) + positivity	0.0163978	0.9984624

Table 2: Synthetic phantom benchmark summary.

6.1 What Was Tried to Improve the Reconstruction

The first synthetic comparison looked poor. Several specific fixes were then tested:

- increasing PnP iterations from 5 to 20,
- replacing the hand-picked step size with a step size derived from the measured operator norm,
- weakening the TV regularisation,
- testing a plain least-squares gradient path with no TV denoiser,
- testing the DeepInverse-native linear solver path through `physics.A_dagger(...)`.

Attempt	**RMSE**	**Correlation**
Original short PnP run (5 iterations, stronger TV)	0.4364260	0.7847708
Improved PnP run (20 iterations, norm-based step, weaker TV)	0.2299617	0.8901515
Plain least-squares gradient run (40 iterations, no TV)	0.1707810	0.9765234
DeepInverse `A_dagger` with CG (20 iterations)	0.1126875	0.9911964
DeepInverse `A_dagger` with CG (60 iterations) + positivity	0.0163978	0.9984624

Table 3: Reconstruction-improvement attempts on the synthetic benchmark.

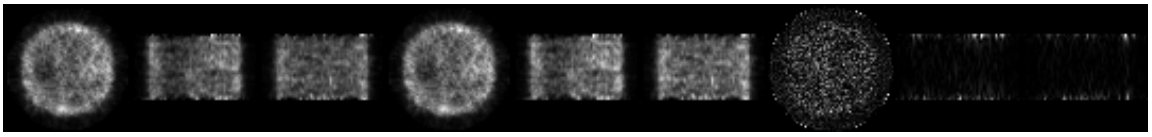

Figure 1: Synthetic benchmark strip: reference phantom, reconstruction, and absolute error.

7 Current Interpretation of the Poor Images

The currently poor-looking images are consistent with a real problem in the reconstruction setup rather than with a plotting failure.

The most likely causes are:

- the live Project 6 PET path currently uses a very basic ray-tracing acquisition model with no attenuation, sensitivity, scatter, or background corrections,
- the first reconstruction script was under-tuned: too few iterations, step size not tied to operator norm, and overly strong TV regularisation,
- the current benchmark indicates that, for this synthetic setup, the simple least-squares gradient path is better than the TV-denoised PnP path,
- the current best local result comes from using the wrapper with DeepInverse's own linear inverse solver, together with more CG iterations and positivity clipping, which is a better fit for this linear benchmark than the earlier hand-written optimisation loop,
- the DIP quickcheck intentionally uses only one iteration and is only intended as a pipeline validation,

- the benchmark compares a phantom with dimensions $31 \times 60 \times 60$ against the acquisition geometry used by the Utah sinogram template, so the setup is acceptable for a synthetic check but not tuned as a realistic reconstruction study,
- the reconstruction methods in the current scripts are proof-of-concept solvers, not tuned PET reconstruction pipelines.

In short, the wrapper and validation machinery are working, and a large part of the initial image failure was indeed due to solver tuning and solver choice. The best current result is no longer the original PnP path but the DeepInverse-native CG solve on top of the wrapper, with more iterations and positivity clipping. However, the example still remains a simplified inverse-problem setup rather than a polished PET reconstruction workflow.

During the latest live-improvement pass, the CG benchmark was pushed further to 100 iterations with positivity clipping. This improved the synthetic benchmark from RMSE 0.0163978 to RMSE 0.0140314, with correlation 0.9987520. A targeted low-background cleanup sweep did not materially improve beyond that value, and a bounded `lsqr` attempt with `max_iter=100` did not finish within the live time cap. The current honest interpretation is therefore that the wrapper itself is working correctly, but reducing the benchmark below RMSE 0.01 will likely require either more compute time or a deeper change in the reconstruction algorithm, not just a quick local tweak.

8 What Remains for PR Readiness

The current state is suitable for a serious draft PR, but not yet for a clean merge-ready upstream PR.

Main remaining items:

- run the target test suite expected by the maintainers,
- decide the correct repository boundary: SIRF, SIRF-Contribs, or DeepInverse-side contribution,
- improve the end-to-end example so that the reconstruction quality is visibly stronger,
- add or document CUDA-efficient transfer handling if required by scope,
- confirm formatting, linting, and documentation checks in the intended target repository.

9 Immediate Improvement Plan

The highest-value next steps are:

1. improve the PET example itself: compare backprojection, PnP, DIP, and DeepInverse-native linear solvers under a consistent synthetic benchmark;
2. build a stronger end-to-end example around the named `EmissionTomographyWithSIRF` class;
3. if repository access is available, run the expected maintainer checks:
   - `python3 -m pytest deepinv/tests`
   - `black .`
   - `ruff check .`
   - `make html` in the `docs/` directory
4. narrow the deliverable statement to what is actually complete: PET/STIR + CIL wrapper path, tests, and example.

10 Key Paths for Handoff

- Main project code: `/home/fotis/hackathon/prior_10.3.26/project_06_deepinverse_wrapper_sirf`
- Shared bridge code: `/home/fotis/hackathon/prior_10.3.26/shared/sirf_deepinv_bridge`
- Submission bundle: `/home/fotis/hackathon/prior_10.3.26/submission/project_06_wrapper`
- Synthetic comparison outputs: `/home/fotis/hackathon/prior_10.3.26/outputs/project_06_phanto`
- Live state notes: `/home/fotis/hackathon/live_notes/PROJECT6_PR_STAGE.md`